



南開大學
Nankai University

计算机学院

并行程序设计调研报告

协程技术调研

从并发演进到底层原理、语言对比与实验验证

作者: kyc

日期: 2026 年 6 月

目录

1	摘要	2
2	并发执行机制为何演进到协程	2
2.1	从进程、线程到事件循环	2
2.2	协程解决的不是“计算速度”，而是“等待方式”	3
3	协程的定义、分类与应用范式	3
3.1	定义与心智模型	3
3.2	有栈协程与无栈协程	4
3.3	三种应用范式	4
4	底层机制：编译器、运行时与操作系统协同	4
4.1	无栈协程：编译器生成状态机	4
4.2	有栈协程：用户态上下文切换	4
4.3	运行时与操作系统的分工	5
5	主流语言支持对比	6
6	实验设计与结果	6
6.1	实验环境与方法	6
6.2	I/O 密集型结果	7
6.3	CPU 密集型与切换开销	8
6.4	内存记录说明	8
7	实验洞察、风险与选型建议	9
7.1	实验洞察	9
7.2	选型建议	9
8	AI 使用说明	9
9	总结	10

1 摘要

随着云服务、网络后端和数据处理系统的并发度不断提高，传统“一个请求一个线程”的编程方式逐渐遇到内存、调度和可维护性瓶颈。协程提供了一种比操作系统线程更轻量的并发抽象：程序员仍然书写接近同步风格的代码，而编译器和运行时在暂停点保存执行状态，并在 I/O 就绪或调度条件满足时恢复执行。

本文围绕“为什么会演进到协程、协程如何实现、主流语言如何支持、协程与线程性能差异如何”四个问题展开。结论可以概括为三点：第一，协程的核心价值不是让单段 CPU 计算更快，而是用更低的内存和调度开销承载大规模 I/O 并发；第二，协程机制需要编译器、语言运行时和操作系统 I/O 多路复用协同完成；第三，语言选型时应关注调度器、取消语义、阻塞调用兼容性和现有生态，而不是只比较“是否支持 `async/await`”。

2 并发执行机制为何演进到协程

2.1 从进程、线程到事件循环

早期服务器常用多进程模型，每个请求或连接由独立进程处理。这种方式隔离性强，但进程地址空间、页表、IPC 和上下文切换成本较高。线程共享地址空间，创建与通信成本低于进程，因此成为长期主流。但线程仍由内核抢占式调度，默认栈空间常以 MB 计；当连接数从数千增长到数万甚至百万时，线程模型会同时受到内存占用、调度队列和锁竞争限制。

事件循环模型通过非阻塞 I/O 与 `epoll`、`kqueue`、IOCP 或 `io_uring` 等机制，将大量连接复用到少量线程上 [3, 1]。它解决了线程数量过多的问题，却把程序逻辑拆成大量回调函数：局部变量、异常传播和控制流都被手工搬运到回调链中，形成所谓“回调地狱”。协程可以理解为对事件循环的抽象封装：底层仍然等待 I/O 事件，表层却恢复为线性的函数书写方式。

表 1: 典型并发模型对比

维度	进程	线程	事件循环 + 回调	协程
调度者	操作系统	操作系统	用户态事件循环	用户态调度器/运行时
切换成本	高	中	低	低
内存成本	地址空间和内核结构	默认栈较大	少量闭包对象	frame 或小栈
编程风格	IPC 明显	同步风格 + 锁	回调嵌套	同步风格 + await/yield
适合场景	强隔离任务	CPU 并行、普通并发	底层网络库	高并发 I/O、异步业务流

2.2 协程解决的不是“计算速度”，而是“等待方式”

协程的暂停点通常出现在 I/O、定时器、channel 收发或显式 yield 处。暂停时，当前协程让出执行权，运行时调度其他可运行协程；事件就绪后再恢复原协程。这一过程避免了“线程阻塞在内核中等待”，也避免了大量线程长期占用栈空间。因此协程最适合高并发 I/O，而不是直接加速 CPU 密集型计算。

3 协程的定义、分类与应用范式

3.1 定义与心智模型

协程可以被看作“可暂停和恢复的函数”。普通函数调用以后必须一路执行到返回，而协程可以在 await、yield 或阻塞点处挂起，保存局部变量、程序计数位置和必要的运行时信息，之后再由调度器恢复。

Listing 1: C++20 风格的协程伪代码

```
1 Task<Response> handle(Request req) {
2     auto user = co_await db.query(req.uid);
3     auto data = co_await rpc.fetch(user);
4     co_return render(data);
5 }
```

上面的代码表面上是顺序执行，实际上每个 co_await 都可能把控制权交还给调度器。与回调写法相比，协程把状态机生成工作交给编译器和运行时，降低了业务代码的心智负担。

3.2 有栈协程与无栈协程

按是否拥有独立执行栈，协程可以分为有栈协程和无栈协程 [2]。

- **有栈协程**：每个协程拥有自己的用户态栈，切换时保存/恢复寄存器和栈指针。优点是可以在较深调用链中挂起，普通函数和协程函数边界更模糊；缺点是需要管理栈空间和栈增长。Go goroutine 与 Java 21 虚拟线程都接近这一类。
- **无栈协程**：协程函数被编译器改写成状态机，局部变量存放在协程 frame 中。优点是内存更小，编译器优化空间大；缺点是函数颜色问题明显，只有被标记的 `async/-coroutine` 函数才能挂起。C++20、Python `asyncio`、C# `async/await`、Rust `async` 都属于这一类。

3.3 三种应用范式

主流协程实践大致可以分为三类。第一类是 **async/await**，以 `Future/Task` 为返回值，适合请求链、RPC 和异步 I/O。第二类是 **CSP/channel**，协程之间通过 `channel` 通信，Go 是典型代表。第三类是 **结构化并发**，父任务持有子任务生命周期，使取消和异常传播更可控。实际工程中，选择范式往往等价于选择错误传播、取消语义和资源回收方式。

4 底层机制：编译器、运行时与操作系统协同

4.1 无栈协程：编译器生成状态机

无栈协程的典型实现是编译器把函数拆成多个状态。每个挂起点对应一个状态编号，局部变量和临时值存放在 `frame` 中，恢复时根据状态跳转到对应位置继续执行。C# 编译器生成 `IAsyncStateMachine`，C++20 编译器生成 `coroutine frame` 和 `coroutine_handle`，Python 的 `async def` 返回 `coroutine` 对象，运行时将其包装为 `Task` 后由事件循环推进 [5]。

这种方式的优势是内存占用可控，很多协程 `frame` 只有几十到几百字节；切换本质上接近一次函数调用或间接跳转。但它也带来函数颜色问题：普通函数不能直接 `await`，同步 API 与异步 API 需要分别维护。

4.2 有栈协程：用户态上下文切换

有栈协程的切换通常包括保存 `callee-saved` 寄存器、保存旧栈指针、切换到新栈并恢复寄存器。简化后的 x86_64 上下文切换可以概括为如下形式：

Listing 2: 有栈协程上下文切换示意

```
1 switch_ctx:
2     push rbp
3     push rbx
4     push r12-r15
5     mov [rdi], rsp ; 保存旧栈指针
6     mov rsp, [rsi] ; 切换到新协程栈
7     pop r15-r12
8     pop rbx
9     pop rbp
10    ret
```

Go 的 goroutine 初始栈很小，并可按需增长；运行时通过 G-M-P 调度模型把大量 goroutine 映射到较少的操作系统线程上。Java 21 虚拟线程则把轻量线程引入 JVM 标准能力，虚拟线程运行在 carrier 平台线程之上，由 JVM 调度并在多数 JDK 阻塞 I/O 处自动让出 carrier[4]。

4.3 运行时与操作系统的分工

协程不是脱离操作系统独立存在的机制。它通常需要三层协作：

1. **编译器或解释器：**识别 `async/await`、`yield` 或 `coroutine` 关键字，生成 frame、状态机和恢复入口。
2. **语言运行时：**维护可运行队列、挂起队列、Task/Future、定时器和调度策略；Go 还包括 work stealing、netpoller 和抢占逻辑 [6]。
3. **操作系统：**提供非阻塞 I/O、多路复用、线程和定时器等基础能力。运行时等待 fd 就绪后，再恢复对应协程。

5 主流语言支持对比

表 2: 主流语言协程能力对比

语言	关键机制	类型	调度器	主要特点
C++20	co_await	无栈	标准不提供	只给底层机制, Task、调度器和 I/O 需库实现
Go	go/channel	有栈	GMP 内建	语法轻, 普通函数可直接作为 goroutine 执行
Python	async/await	无栈	单线程事件循环	适合 I/O; CPU 密集受 GIL 和单线程循环限制
C#	async/await	无栈	线程池/上下文	工程化成熟, UI/服务端生态完善
Java 21	Virtual Thread	有栈	JVM 调度	保持同步代码风格, 降低旧系统迁移成本
Rust	async/await	无栈	第三方运行时	零成本抽象强, 但生态和生命周期学习成本高

值得注意的是, C++20 和 Rust 都把“协程语法”与“运行时”分离, 因此同样的 async 代码在不同 executor 上表现可能不同。Go 和 Java 21 则更强调运行时内建调度, 把开发者从 executor 选择中解放出来。Python asyncio 上手简单, 但所有阻塞同步调用都会阻塞事件循环; C# async/await 则展示了无栈状态机在工业语言中的成熟形态。

6 实验设计与结果

6.1 实验环境与方法

实验在同一台本机上运行, 硬件与软件环境如下:

- CPU: 13th Gen Intel(R) Core(TM) i9-13900H, 14 核 20 逻辑处理器;
- 操作系统: Windows 11 Home China 64-bit, 版本 10.0.26200;

- Python: 3.12.9; Go: 1.25.6; Java: OpenJDK 21.0.5;
- 图表生成: `micromamba activate test` 后执行 `python plot_results.py`。

测试分为三类: I/O 密集型测试创建 10000 个任务, 每个任务 `sleep 100 ms`, 用于模拟大量网络等待; CPU 密集型测试创建 1000 个任务, 每个任务计算 `fib(25)`; 上下文切换测试使用大量 `yield/sleep(0)` 或轻量任务估计调度开销。完整代码见同目录下 `bench.py`、`bench.go` 和 `Bench.java`。

6.2 I/O 密集型结果

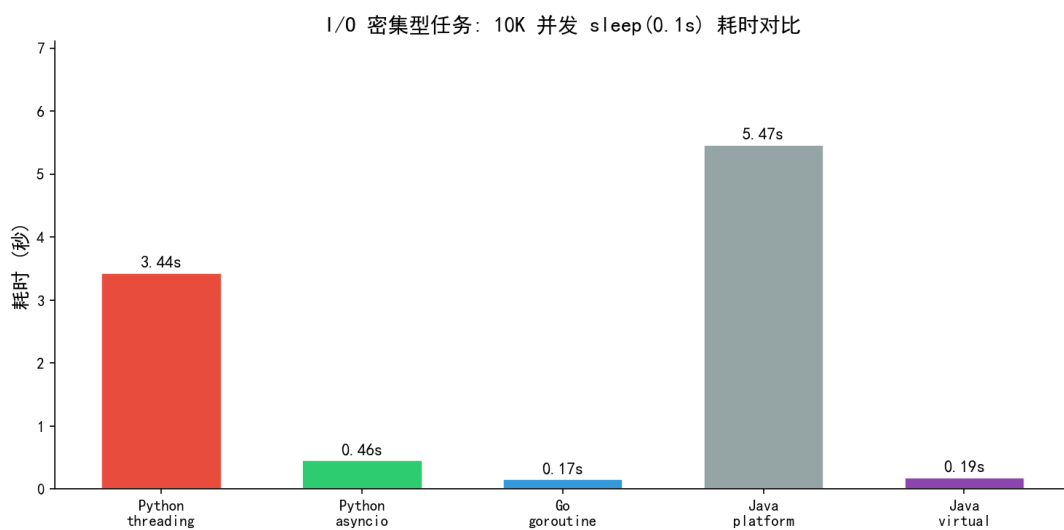


图 1: 10000 个 I/O 等待任务耗时对比

表 3: I/O 密集型实验结果

模型	10000 任务耗时	相对线程基线
Python threading	3.435 s	1.0×
Python asyncio	0.464 s	7.4×
Go goroutine	0.166 s	20.7×
Java platform thread (200 固定线程池)	5.467 s	1.0×
Java virtual thread	0.191 s	28.6×

结果符合预期: I/O 密集场景中, 协程/虚拟线程能在等待期间释放执行权, 使少量内核线程服务大量逻辑任务。Python asyncio 相比 Python threading 快约 7.4 倍; Go goroutine 和 Java 虚拟线程都能在 0.2 秒量级完成 10000 个 `sleep` 任务, 明显优于基于平台线程的对照。

6.3 CPU 密集型与切换开销

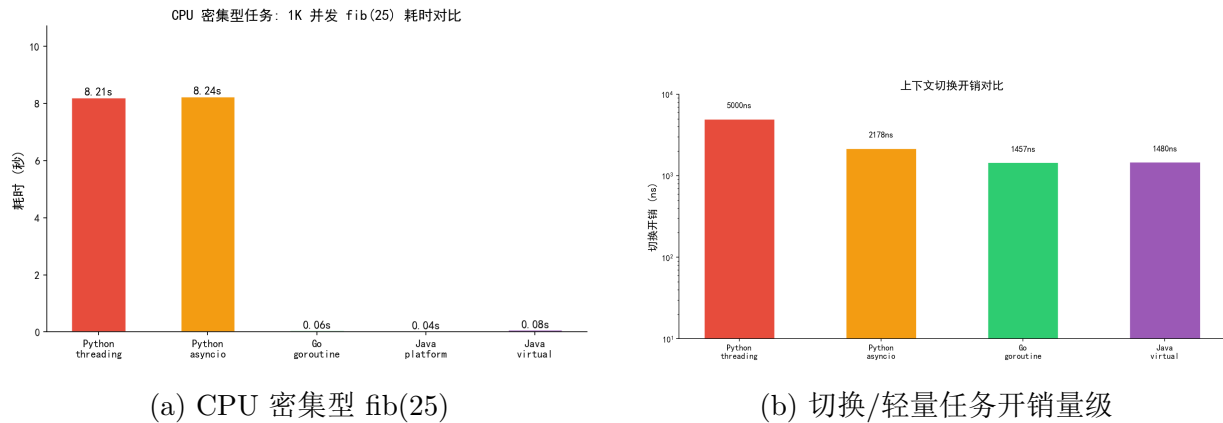


图 2: CPU 密集型和调度开销实验

Python 的 `threading` 与 `asyncio` 在 CPU 密集型测试中耗时几乎相同，说明协程并不会减少计算本身的 CPU 成本。Go 和 Java 的 CPU 测试更快，主要原因不是“协程让计算变快”，而是运行时可以把任务分配到多核上执行。Java 平台线程池在该测试中略快于虚拟线程，也说明虚拟线程的优势不在短小 CPU 任务，而在大量阻塞等待任务。

切换开销图使用对数坐标展示量级差异。Python `asyncio` 的百万次 `yield` 约为 2.178 秒，Go `goroutine` 的 100000 次 `channel` 交互约为 145.7 ms，Java 虚拟线程的 100000 个 `sleep(0)` 任务约为 148 ms。由于不同语言的微基准并不完全等价，这些数字更适合用于判断量级，而不应作为绝对排名。

6.4 内存记录说明

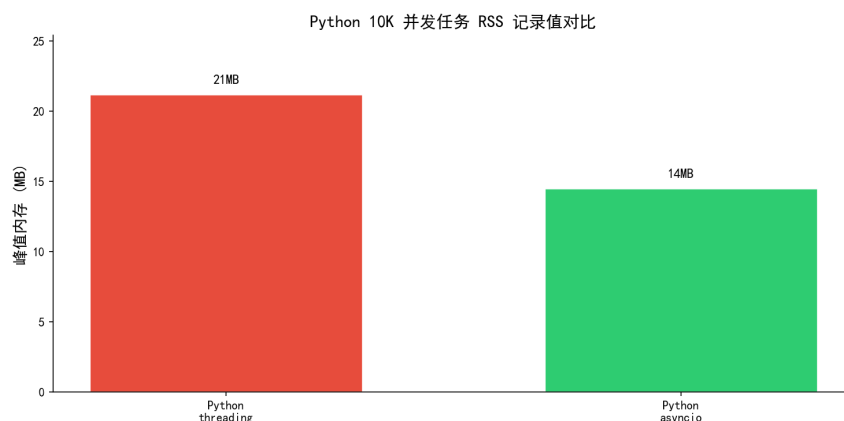


图 3: Python 进程 RSS 记录值对比

本次 Python 脚本记录的是任务结束附近的进程 RSS，而不是统一采样得到的跨语言峰值 RSS，因此只用于辅助说明 `asyncio` 的对象与栈开销较低，报告不把它作为严格

内存排名依据。若要进一步精确比较，应使用外部采样器或平台一致的监控工具记录峰值 RSS，并在 Java/Go 中补充 GC 前后内存统计。

7 实验洞察、风险与选型建议

7.1 实验洞察

1. **I/O 密集任务中协程优势显著。**大量任务处于等待态时，协程通过用户态调度避免大量内核线程阻塞，吞吐和资源利用率明显优于传统线程池。
2. **CPU 密集任务不应期待协程自动加速。**如果瓶颈是计算，应使用多线程、多进程、SIMD 或 GPU；协程只能减少等待和调度成本。
3. **阻塞调用会破坏协程模型。**在 Python async 函数中调用同步 requests，在 Java 虚拟线程中进入长时间 synchronized/JNI 区域，都会使运行时难以释放底层执行资源。
4. **取消与生命周期管理同样重要。**大量协程如果没有结构化管理，可能变成任务泄漏、资源泄漏或异常被吞掉。

7.2 选型建议

如果项目瓶颈在 CPU，优先考虑并行计算手段，而不是协程。如果瓶颈在 I/O 且并发量较小，传统线程池已经足够；当并发量达到数千以上时，再考虑协程或虚拟线程。新建高并发后端服务可以优先考虑 Go goroutine；JVM 旧系统希望保留同步代码风格时，Java 21 虚拟线程很有吸引力；Python Web 或爬虫任务适合 asyncio/FastAPI/uvloop，但必须严格避免阻塞调用；系统级高性能服务可以考虑 Rust tokio 或 C++20 协程与 io_uring/asio 组合。

8 AI 使用说明

按照作业要求，本次调研和材料制作中使用 AI 的方式如下：

1. 使用 AI 辅助梳理“并发演进-协程原理-语言对比-实验验证-选型建议”的讲述结构，再人工调整顺序和重点。
2. 使用 AI 生成和校对部分示意图、表格和代码片段的表达，但实验代码在本机运行后用真实数据替换。
3. 使用 AI 帮助检查术语一致性，例如有栈/无栈协程、函数颜色、GMP 调度器、virtual thread pinning 等。

4. 对关键版本号、JEP/PEP、语言特性和运行时机制回查官方文档或源代码说明，避免把 AI 输出直接当作事实。
5. 最终 Slides 与报告文字均围绕本人实验和理解重新组织，避免大段复制资料原文。

9 总结

协程的本质不是“神奇的快速线程”，而是把等待态任务从昂贵的内核线程中解放出来，用编译器状态机、用户态调度器和操作系统 I/O 多路复用共同支撑高并发。它让程序员可以用近似同步的代码表达异步控制流，是云时代服务端编程的重要抽象。

从技术趋势看，协程正在朝两个方向发展：一类是 C++20/Rust/Python/C# 代表的无栈 `async/await`，强调精细控制和低内存；另一类是 Go goroutine 与 Java 21 虚拟线程代表的有栈轻量线程，强调普通同步代码的可扩展性。面对具体工程问题时，最关键的判断不是“哪种协程更先进”，而是瓶颈是否在 I/O、阻塞调用能否被运行时接管、错误和取消能否被结构化管理。

参考文献

- [1] Jens Axboe. io_uring - linux kernel async i/o interface. https://kernel.dk/io_uring.pdf.
- [2] Ana Lúcia de Moura and Roberto Ierusalimsky. Revisiting coroutines. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 31(2):1–31, 2009.
- [3] Linux man-pages project. epoll - i/o event notification facility. <https://man7.org/linux/man-pages/man7/epoll.7.html>.
- [4] Oracle. Jep 444: Virtual threads. <https://openjdk.org/jeps/444>, 2023.
- [5] Yury Selivanov. Pep 492 – coroutines with async and await syntax. <https://peps.python.org/pep-0492/>, 2015.
- [6] The Go Authors. Go runtime source - runtime/proc.go. <https://github.com/golang/go/blob/master/src/runtime/proc.go>.